17.4 上下文压缩（Compaction）

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~450,000 tokens，输出 ~36,000 tokens（本章合计）

大语言模型有一个固有限制——**上下文窗口**（Context Window）。无论模型的上下文窗口是 8K、128K 还是 200K token，长时间对话终将耗尽可用空间。当对话历史的 token 数接近上下文窗口上限时，如果不做处理，模型 API 会返回"上下文溢出"错误，对话将被迫中断。

OpenClaw 的**上下文压缩（Compaction）机制优雅地解决了这个问题：在上下文溢出发生前，将旧的对话历史摘要化**——用一段简短的总结替代大量的原始消息，从而释放上下文空间，让对话可以无限期持续。

> **衍生解释——上下文窗口（Context Window）**
>
> 上下文窗口是大语言模型在一次推理中能"看到"的最大文本量，以 token 为单位。一个 token 大约对应 4 个英文字符或 1-2 个中文字符。例如 Claude 的上下文窗口为 200K token，意味着一次对话中的所有消息（包括系统提示词）总计不能超过约 200,000 个 token。超出这个限制，API 会拒绝请求。

17.4.1 自动压缩触发机制

触发时机：上下文溢出检测

OpenClaw 不会预防性地定期压缩——它采用**被动触发**策略：当模型 API 返回上下文溢出错误时，才启动压缩流程。

在 `runEmbeddedPiAgent` 的主循环中（参见 7.2 节），每次 API 调用后都会检查错误类型：

```
// src/agents/pi-embedded-runner/run.ts (简化)
const MAX_OVERFLOW_COMPACTION_ATTEMPTS = 3;
let overflowCompactionAttempts = 0;

while (true) {
  const attempt = await runAttempt({ ... });

  if (promptError && !aborted) {
    const errorText = describeUnknownError(promptError);

    if (isContextOverflowError(errorText)) {
      const isCompactionFailure = isCompactionFailureError(errorText);

      // 仅对"真正的"上下文溢出进行压缩，不对压缩失败再次压缩
      if (!isCompactionFailure &&
          overflowCompactionAttempts < MAX_OVERFLOW_COMPACTION_ATTEMPTS) {
        overflowCompactionAttempts++;
        log.warn(
          `context overflow detected (attempt ${overflowCompactionAttempts}/` +
          `${MAX_OVERFLOW_COMPACTION_ATTEMPTS}); attempting auto-compaction`
        );

        const compactResult = await compactEmbeddedPiSessionDirect({
          sessionId: params.sessionId,
          sessionFile: params.sessionFile,
          workspaceDir: resolvedWorkspace,
          provider,
          model: modelId,
          // ...其他参数
        });

        if (compactResult.compacted) {
          log.info("auto-compaction succeeded; retrying prompt");
          continue; // 压缩成功，重试当前请求
        }

        log.warn(`auto-compaction failed: ${compactResult.reason}`);
      }

      // 所有压缩尝试失败或不可压缩
      return {
        payloads: [{
          text: "Context overflow: prompt too large for the model.",
```

```
          isError: true,
        }],
        meta: { error: { kind: isCompactionFailure
          ? "compaction_failure" : "context_overflow" } },
      };
    }
  }
  // ...正常处理
}
```

几个关键设计决策：

1. **最多 3 次压缩重试**： `MAX_OVERFLOW_COMPACTION_ATTEMPTS = 3` ，防止在压缩效果不足时陷入无限循环。
2. **区分溢出和压缩失败**：如果错误本身就是"压缩失败导致的溢出"（ `compaction_failure` ），不会再次尝试压缩，避免递归问题。
3. **成功后重试**：压缩成功后通过 `continue` 直接重试当前请求，用户无感知。

两种压缩模式

OpenClaw 支持两种压缩模式，通过配置选择：

```
// src/agents/pi-embedded-runner/extensions.ts
function resolveCompactionMode(cfg?: OpenClawConfig): "default" | "safeguard" {
  return cfg?.agents?.defaults?.compaction?.mode === "safeguard"
    ? "safeguard" : "default";
}
```

模式	实现	特点
`"default"`	Pi SDK 内置的 `session.compact()`	简单直接
`"safeguard"`	`compaction-safeguard` 扩展	自适应分块、历史剪裁、工具失败保留

"safeguard" 模式是更高级的压缩策略，下面会详细介绍。

17.4.2 压缩流程（`src/agents/compaction.ts`）

核心算法：Token 估算与消息分块

压缩的第一步是估算消息的 token 数量：

```
// src/agents/compaction.ts
export function estimateMessagesTokens(messages: AgentMessage[]): number {
  return messages.reduce(
    (sum, message) => sum + estimateTokens(message), 0
  );
}
```

按 Token 份额分割消息

`splitMessagesByTokenShare` 将消息按 token 数量等分为多个块，每块的 token 数大致相等：

```
export function splitMessagesByTokenShare(
  messages: AgentMessage[],
  parts = 2,  // 默认分成 2 块
): AgentMessage[][] {
  if (messages.length === 0) return [];

  const normalizedParts = normalizeParts(parts, messages.length);
  if (normalizedParts <= 1) return [messages];

  const totalTokens = estimateMessagesTokens(messages);
  const targetTokens = totalTokens / normalizedParts;
  const chunks: AgentMessage[][] = [];
  let current: AgentMessage[] = [];
  let currentTokens = 0;

  for (const message of messages) {
    const messageTokens = estimateTokens(message);
    // 当前块达到目标大小且还有剩余块 → 开始新块
    if (chunks.length < normalizedParts - 1 &&
        current.length > 0 &&
        currentTokens + messageTokens > targetTokens) {
      chunks.push(current);
      current = [];
      currentTokens = 0;
    }
    current.push(message);
    currentTokens += messageTokens;
  }

  if (current.length > 0) chunks.push(current);
  return chunks;
}
```

按最大 Token 数分块

另一种分块方式是设置每块的最大 token 上限：

```typescript
export function chunkMessagesByMaxTokens(
  messages: AgentMessage[],
  maxTokens: number,
): AgentMessage[][] {
  const chunks: AgentMessage[][] = [];
  let currentChunk: AgentMessage[] = [];
  let currentTokens = 0;

  for (const message of messages) {
    const messageTokens = estimateTokens(message);
    if (currentChunk.length > 0 &&
        currentTokens + messageTokens > maxTokens) {
      chunks.push(currentChunk);
      currentChunk = [];
      currentTokens = 0;
    }
    currentChunk.push(message);
    currentTokens += messageTokens;

    // 处理超大消息：单独成块
    if (messageTokens > maxTokens) {
      chunks.push(currentChunk);
      currentChunk = [];
      currentTokens = 0;
    }
  }

  if (currentChunk.length > 0) chunks.push(currentChunk);
  return chunks;
}
```

自适应分块比率

当消息平均体积较大时，需要使用更小的分块比率来避免单个分块超出模型限制：

```
export const BASE_CHUNK_RATIO = 0.4;     // 基础：上下文窗口的 40%
export const MIN_CHUNK_RATIO = 0.15;      // 最小：15%
export const SAFETY_MARGIN = 1.2;         // 20% 安全裕量

export function computeAdaptiveChunkRatio(
  messages: AgentMessage[],
  contextWindow: number
): number {
  if (messages.length === 0) return BASE_CHUNK_RATIO;

  const totalTokens = estimateMessagesTokens(messages);
  const avgTokens = totalTokens / messages.length;

  // 加上安全裕量（token 估算可能偏低）
  const safeAvgTokens = avgTokens * SAFETY_MARGIN;
  const avgRatio = safeAvgTokens / contextWindow;

  // 如果平均消息大于上下文的 10%，降低分块比率
  if (avgRatio > 0.1) {
    const reduction = Math.min(
      avgRatio * 2,
      BASE_CHUNK_RATIO - MIN_CHUNK_RATIO
    );
    return Math.max(MIN_CHUNK_RATIO, BASE_CHUNK_RATIO - reduction);
  }

  return BASE_CHUNK_RATIO;
}
```

举例说明：

- **普通对话**（平均消息 ~500 token，200K 上下文）： `avgRatio = 0.003` ，远小于 10%，使用基础比率 40%
- **含大型代码块**（平均消息 ~30K token，200K 上下文）： `avgRatio = 0.18` ，使用降低后的比率 `max(0.15, 0.4 - 0.36) = 0.15`

超大消息检测

如果单条消息就超过上下文窗口的 50%，则无法安全摘要化：

```
export function isOversizedForSummary(
  msg: AgentMessage,
  contextWindow: number
): boolean {
  const tokens = estimateTokens(msg) * SAFETY_MARGIN;
  return tokens > contextWindow * 0.5;
}
```

分阶段摘要（summarizeInStages）

这是压缩的核心函数。它先将消息分块，对每块分别生成摘要，最后将所有摘要合并为一份综合摘要：

```typescript
export async function summarizeInStages(params: {
  messages: AgentMessage[];
  model: NonNullable<ExtensionContext["model"]>;
  apiKey: string;
  signal: AbortSignal;
  reserveTokens: number;
  maxChunkTokens: number;
  contextWindow: number;
  customInstructions?: string;
  previousSummary?: string;
  parts?: number;              // 分块数，默认 2
  minMessagesForSplit?: number; // 最少消息数才分块，默认 4
}): Promise<string> {
  const { messages } = params;
  if (messages.length === 0) {
    return params.previousSummary ?? "No prior history.";
  }

  // 消息太少或 token 数在限制内 → 直接摘要
  const parts = normalizeParts(params.parts ?? 2, messages.length);
  const totalTokens = estimateMessagesTokens(messages);
  if (parts <= 1 ||
      messages.length < minMessagesForSplit ||
      totalTokens <= params.maxChunkTokens) {
    return summarizeWithFallback(params);
  }

  // 分块摘要
  const splits = splitMessagesByTokenShare(messages, parts)
    .filter((chunk) => chunk.length > 0);
  if (splits.length <= 1) {
    return summarizeWithFallback(params);
  }

  // 逐块生成摘要
  const partialSummaries: string[] = [];
  for (const chunk of splits) {
    partialSummaries.push(
      await summarizeWithFallback({
        ...params,
        messages: chunk,
        previousSummary: undefined,
      })
```

```
    );
  }

  if (partialSummaries.length === 1) return partialSummaries[0];

  // 合并所有部分摘要
  const summaryMessages: AgentMessage[] = partialSummaries.map(
    (summary) => ({ role: "user", content: summary, timestamp: Date.now() })
  );

  const mergeInstructions =
    "Merge these partial summaries into a single cohesive summary. " +
    "Preserve decisions, TODOs, open questions, and any constraints.";

  return summarizeWithFallback({
    ...params,
    messages: summaryMessages,
    customInstructions: mergeInstructions,
  });
}
```

流程图:

```
原始消息 [M1, M2, M3, ..., Mn]
          |
    splitMessagesByTokenShare(2)
          |
    ┌─────┴─────┐
    |           |
 块 A         块 B
 [M1..Mk]    [Mk+1..Mn]
    |           |
 summarize   summarize
    |           |
 摘要 A       摘要 B
    |           |
    └─────┬─────┘
          |
    合并摘要请求
    "Merge these partial summaries..."
          |
       最终摘要
```

渐进式降级（summarizeWithFallback）

如果完整摘要化失败（例如消息太大），系统会尝试降级策略：

```
export async function summarizeWithFallback(params: {
  messages: AgentMessage[];
  model: NonNullable<ExtensionContext["model"]>;
  // ...
}): Promise<string> {
  // 尝试 1：完整摘要
  try {
    return await summarizeChunks(params);
  } catch (fullError) {
    console.warn("Full summarization failed, trying partial...");
  }

  // 尝试 2：仅摘要小消息，跳过超大消息
  const smallMessages: AgentMessage[] = [];
  const oversizedNotes: string[] = [];

  for (const msg of messages) {
    if (isOversizedForSummary(msg, contextWindow)) {
      const tokens = estimateTokens(msg);
      oversizedNotes.push(
        `[Large ${role} (~${Math.round(tokens/1000)}K tokens) ` +
        `omitted from summary]`
      );
    } else {
      smallMessages.push(msg);
    }
  }

  if (smallMessages.length > 0) {
    try {
      const partialSummary = await summarizeChunks({
        ...params, messages: smallMessages
      });
      const notes = oversizedNotes.join("\n");
      return partialSummary + (notes ? `\n\n${notes}` : "");
    } catch {
      console.warn("Partial summarization also failed.");
    }
  }

  // 最终降级：仅记录存在的消息数
  return `Context contained ${messages.length} messages ` +
    `(${oversizedNotes.length} oversized). ` +
```

```
    `Summary unavailable due to size limits.`;
}
```

三级降级策略：

级别	策略	何时触发
1	完整摘要	默认尝试
2	部分摘要（跳过超大消息）	完整摘要失败
3	统计信息	部分摘要也失败

17.4.3 压缩前的记忆刷新（Memory Flush）

在压缩之前，需要确保所有未保存的工具调用结果已经写入会话记录。这是因为压缩会**丢弃**原始消息，如果工具结果未保存，这些信息将永远丢失。

`compact.ts` 中的 `compactEmbeddedPiSessionDirect` 函数在压缩结束后执行刷新：

```typescript
// src/agents/pi-embedded-runner/compact.ts（简化）
export async function compactEmbeddedPiSessionDirect(
  params: CompactEmbeddedPiSessionParams,
): Promise<EmbeddedPiCompactResult> {
  // ...模型解析、API Key 获取等准备工作

  const sessionLock = await acquireSessionWriteLock({
    sessionFile: params.sessionFile,
  });

  try {
    // 修复可能损坏的会话文件
    await repairSessionFileIfNeeded({ sessionFile: params.sessionFile });

    // 创建 Agent 会话
    const { session } = await createAgentSession({ ... });

    // 应用系统提示词覆盖
    applySystemPromptOverrideToSession(session, systemPromptOverride());

    try {
      // 清理和验证对话历史
      const sanitized = await sanitizeSessionHistory({ ... });
      const validated = validateAnthropicTurns(
        validateGeminiTurns(sanitized)
      );
      const limited = limitHistoryTurns(validated, historyLimit);

      if (limited.length > 0) {
        session.agent.replaceMessages(limited);
      }

      // 执行压缩
      const result = await session.compact(params.customInstructions);

      // 估算压缩后的 token 数
      let tokensAfter: number | undefined;
      try {
        tokensAfter = 0;
        for (const message of session.messages) {
          tokensAfter += estimateTokens(message);
        }
        if (tokensAfter > result.tokensBefore) {
```

```
          tokensAfter = undefined; // 估算不可信
        }
      } catch {
        tokensAfter = undefined;
      }

      return {
        ok: true,
        compacted: true,
        result: {
          summary: result.summary,
          firstKeptEntryId: result.firstKeptEntryId,
          tokensBefore: result.tokensBefore,
          tokensAfter,
        },
      };
    } finally {
      // 关键：刷新待写入的工具结果
      sessionManager.flushPendingToolResults?.();
      session.dispose();
    }
  } finally {
    await sessionLock.release();
  }
}
```

会话写锁

压缩操作需要独占访问会话文件，通过 `acquireSessionWriteLock` 实现：

```
const sessionLock = await acquireSessionWriteLock({
  sessionFile: params.sessionFile,
});
```

> **衍生解释——写锁（Write Lock）**
>
> 写锁是一种并发控制机制：当一个操作持有写锁时，其他操作不能同时读写同一资源。OpenClaw 在压缩期间获取写锁，防止其他线程（如用户消息处理）同时修改会话文件，从而避免数据竞争和文件损坏。

历史清理流程

在压缩前，OpenClaw 对对话历史执行一系列清理操作：

```
原始消息 → sanitizeSessionHistory → validateGeminiTurns
  → validateAnthropicTurns → limitHistoryTurns → 压缩
```

1. **sanitizeSessionHistory**：移除不完整的消息、清理图片数据等
2. **validateGeminiTurns**：确保 Gemini 模型要求的"用户先说"顺序
3. **validateAnthropicTurns**：确保 Anthropic 模型的 user/assistant 交替顺序
4. **limitHistoryTurns**：按配置限制最大对话轮数

Lane 排队

压缩可以在两种上下文中调用：

```
// 方式 1：直接调用（已在 Lane 内部）
export async function compactEmbeddedPiSessionDirect(params) { ... }

// 方式 2：通过 Lane 排队（外部调用）
export async function compactEmbeddedPiSession(params) {
  const sessionLane = resolveSessionLane(params.sessionKey || params.sessionId);
  const globalLane = resolveGlobalLane(params.lane);
  return enqueueCommandInLane(sessionLane, () =>
    enqueueGlobal(async () => compactEmbeddedPiSessionDirect(params))
  );
}
```

`compactEmbeddedPiSession` 通过**双重 Lane 排队**（会话 Lane + 全局 Lane）确保压缩操作不会与同一会话的其他操作冲突，也不会超出全局并发限制。 `compactEmbeddedPiSessionDirect` 则用于已经在 Lane 内部的场景（如自动压缩重试），避免死锁。

17.4.4 压缩重试与缓冲重置

Safeguard 模式：高级压缩策略

"safeguard" 模式通过 `compaction-safeguard` 扩展实现了更智能的压缩：

```typescript
// src/agents/pi-extensions/compaction-safeguard.ts（简化）
export default function compactionSafeguardExtension(api: ExtensionAPI): void {
  api.on("session_before_compact", async (event, ctx) => {
    const { preparation, customInstructions, signal } = event;
    const { readFiles, modifiedFiles } = computeFileLists(preparation.fileOps);
    const toolFailures = collectToolFailures([
      ...preparation.messagesToSummarize,
      ...preparation.turnPrefixMessages,
    ]);

    // 构建降级摘要（无模型时使用）
    const fallbackSummary =
      `${FALLBACK_SUMMARY}${toolFailureSection}${fileOpsSummary}`;

    const model = ctx.model;
    const apiKey = await ctx.modelRegistry.getApiKey(model);
    if (!model || !apiKey) {
      return { compaction: { summary: fallbackSummary, ... } };
    }

    // 计算上下文预算
    const contextWindowTokens = runtime?.contextWindowTokens ?? modelContextWindow;
    const maxHistoryShare = runtime?.maxHistoryShare ?? 0.5;

    // 如果新内容（系统提示 + 工具 Schema）占用过多空间，
    // 先裁剪历史消息
    if (newContentTokens > maxHistoryTokens) {
      const pruned = pruneHistoryForContextShare({
        messages: messagesToSummarize,
        maxContextTokens: contextWindowTokens,
        maxHistoryShare,
        parts: 2,
      });

      if (pruned.droppedChunks > 0) {
        // 对被裁剪的消息也生成摘要
        droppedSummary = await summarizeInStages({
          messages: pruned.droppedMessagesList,
          ...
        });
        messagesToSummarize = pruned.messages;
      }
    }
```

```
    // 主摘要：自适应分块
    const adaptiveRatio = computeAdaptiveChunkRatio(
      allMessages, contextWindowTokens
    );
    const historySummary = await summarizeInStages({
      messages: messagesToSummarize,
      maxChunkTokens: Math.floor(contextWindowTokens * adaptiveRatio),
      previousSummary: droppedSummary, // 包含裁剪消息的摘要
      ...
    });

    // 附加工具失败信息和文件操作记录
    summary += toolFailureSection;
    summary += fileOpsSummary;

    return { compaction: { summary, ... } };
  });
}
```

Safeguard 模式的核心改进：

1. 历史裁剪（History Pruning）

当系统提示词 + 工具 Schema 已经占用了大量上下文时，留给对话历史的空间可能不足。 `pruneHistoryForContextShare` 会迭代地裁剪最旧的消息块：

```
export function pruneHistoryForContextShare(params: {
  messages: AgentMessage[];
  maxContextTokens: number;
  maxHistoryShare?: number; // 默认 0.5 (50%)
  parts?: number;
}): {
  messages: AgentMessage[];        // 保留的消息
  droppedMessagesList: AgentMessage[]; // 被裁剪的消息
  droppedChunks: number;
  droppedMessages: number;
  droppedTokens: number;
  keptTokens: number;
  budgetTokens: number;
} {
  const budgetTokens = Math.floor(maxContextTokens * maxHistoryShare);
  let keptMessages = params.messages;

  while (keptMessages.length > 0 &&
         estimateMessagesTokens(keptMessages) > budgetTokens) {
    const chunks = splitMessagesByTokenShare(keptMessages, parts);
    if (chunks.length <= 1) break;

    const [dropped, ...rest] = chunks;
    const flatRest = rest.flat();

    // 修复工具调用配对
    const repairReport = repairToolUseResultPairing(flatRest);
    keptMessages = repairReport.messages;

    allDroppedMessages.push(...dropped);
    droppedChunks++;
  }

  return { messages: keptMessages, droppedMessagesList: allDroppedMessages, ... };
}
```

> **衍生解释——工具调用配对修复**
>
> 在 Anthropic 的 API 中，每个 `tool_use` （工具调用）必须有对应的 `tool_result` （工具结果），反之亦然。当裁剪历史消息时，可能会出现"孤儿" `tool_result` ——它对应的 `tool_use` 在被裁剪的块中。 `repairToolUseResultPairing` 会检测并移除这些孤儿，防止 API 返回 "unexpected tool_use_id" 错误。

2. 工具失败保留

压缩时不仅保留对话摘要，还特别保留了**工具执行失败**的信息：

```
function collectToolFailures(
  messages: AgentMessage[]
): ToolFailure[] {
  const failures: ToolFailure[] = [];
  for (const message of messages) {
    if (role !== "toolResult" || toolResult.isError !== true) continue;

    failures.push({
      toolCallId,
      toolName,
      summary: truncateFailureText(normalizedText, 240),
      meta: formatToolFailureMeta(toolResult.details),
    });
  }
  return failures;
}
```

这些失败信息以结构化格式附加在摘要末尾：

```
## Tool Failures
- exec (exitCode=1): npm test failed with 3 errors
- write: Permission denied: /etc/hosts
```

3. 文件操作记录

压缩摘要还包含会话期间读取和修改的文件列表：

```
function computeFileLists(fileOps: FileOperations): {
  readFiles: string[];
  modifiedFiles: string[];
} {
  const modified = new Set([...fileOps.edited, ...fileOps.written]);
  const readFiles = [...fileOps.read]
    .filter((f) => !modified.has(f)).toSorted();
  const modifiedFiles = [...modified].toSorted();
  return { readFiles, modifiedFiles };
}
```

输出格式：

```
<read-files>
src/agents/system-prompt.ts
src/config/config.ts
</read-files>

<modified-files>
src/agents/identity.ts
tests/identity.test.ts
</modified-files>
```

压缩事件的订阅与缓冲重置

从用户可见性的角度，压缩期间需要暂停消息流的推送，并在压缩完成后重置缓冲区：

```
// src/agents/pi-embedded-subscribe.handlers.lifecycle.ts（简化）
case "auto_compaction_start":
  ctx.state.compactionInFlight = true;
  ctx.emit({ stream: "compaction", type: "start" });
  break;

case "auto_compaction_end":
  ctx.state.compactionInFlight = false;
  if (event.willRetry) {
    // 压缩完成但会重试 → 等待下一轮
    ctx.emit({ stream: "compaction", type: "retry" });
  } else {
    // 压缩彻底完成 → 解除阻塞
    ctx.emit({ stream: "compaction", type: "end" });
  }
  break;
```

订阅端通过 `compactionRetryPromise` 等待压缩完成：

```
// src/agents/pi-embedded-subscribe.ts（简化）
isCompacting: () =>
  state.compactionInFlight || state.pendingCompactionRetry > 0,

waitForCompaction: () => {
  if (state.compactionInFlight || state.pendingCompactionRetry > 0) {
    return state.compactionRetryPromise ?? Promise.resolve();
  }
}
```

这确保了：

- 压缩期间，前端不会收到不完整的文本流
- 压缩重试时，缓冲区被正确重置
- 压缩完成后，新的文本流从干净的状态开始

压缩结果类型

```
// src/agents/pi-embedded-runner/types.ts
export type EmbeddedPiCompactResult = {
  ok: boolean;
  compacted: boolean;
  reason?: string;
  result?: {
    summary: string;
    firstKeptEntryId: string;
    tokensBefore: number;
    tokensAfter?: number;
    details?: { readFiles: string[]; modifiedFiles: string[] };
  };
};
```

一次成功的压缩可能产生如下结果：

```
{
  "ok": true,
  "compacted": true,
  "result": {
    "summary": "User asked to implement a REST API for user authentication. We created
    "tokensBefore": 185000,
    "tokensAfter": 12000,
    "details": {
      "readFiles": ["src/api/auth.ts", "src/config/db.ts"],
      "modifiedFiles": ["src/api/auth.ts", "tests/auth.test.ts"]
    }
  }
}
```

在这个例子中，压缩将 185K token 的对话历史压缩为约 12K token 的摘要，释放了 93% 的上下文空间。

本节小结

1. **上下文压缩**是 OpenClaw 实现"无限对话"的核心机制——通过将旧消息摘要化来释放上下文窗口空间。
2. **触发策略**是被动式的：仅在模型 API 返回上下文溢出错误时才启动，最多重试 3 次，压缩失败本身不会再触发压缩。
3. **分阶段摘要**（summarizeInStages）将消息分块后逐块摘要，再合并为最终摘要，有效处理超大上下文。
4. **自适应分块**（computeAdaptiveChunkRatio）根据消息平均大小动态调整分块比率，避免超出模型限制。
5. **三级降级**策略（完整摘要 → 部分摘要 → 统计信息）确保压缩流程不会因少数超大消息而完全失败。
6. **Safeguard 模式**增加了历史裁剪、工具失败保留、文件操作记录等增强功能，生成更有信息量的压缩摘要。
7. **并发安全**通过写锁和 Lane 排队保证压缩期间会话文件不被并发修改。